

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Московский Авиационный Институт»
(Национальный Исследовательский Университет)

Институт: №8 «Информационные технологии
и прикладная математика»
Кафедра: 806 «Вычислительная математика
и программирование»

Лабораторная работа № 2
по курсу «Базы данных»

Группа: М8О-307Б-22

Студент(ка): П. В. Лебедько

Преподаватель: А. В. Малахов

Оценка:

Дата: 09.05.2025

Москва, 2025

ОГЛАВЛЕНИЕ

1	Задание	3
2	Ход работы.....	3
3	Выводы.....	9

1 Задание

1. Выделить данные, требующие быстрого доступа, временного хранения или частого обновления, включая:

- Токены авторизации пользователей
- Сессионные данные
- Часто запрашиваемые или временные данные
- Данные, требующие мгновенного обновления (статусы заказов, уведомления)

2. Для токенов авторизации:

- Реализовать хранение в Redis с настройкой TTL (время жизни) через конфигурацию приложения
- Обеспечить автоматическое удаление просроченных токенов средствами Redis

3. Для остальных динамических данных реализовать оптимальный формат хранения в Redis, обосновав выбор структуры для конкретных сценариев использования.

4. Настроить мгновенные оповещения о критичных событиях (например, новые заказы, изменения статусов) через PubSub

5. Объяснить, как и почему должна измениться скорость работы системы после внедрения redis

2 Ход работы

С Redis будем работать, используя Redis Sentinel, как того требует userver:

```
redis-sentinel:
  image: redis:latest
  command: redis-sentinel /usr/local/etc/redis/sentinel.conf
  ports:
    - "26379:26379"
  volumes:
    - ${PWD}/configs/sentinel.conf:/usr/local/etc/redis/sentinel.conf
  depends_on:
    - redis-master
  networks:
    - redis

redis-master:
  image: redis:latest
  command: redis-server --requirepass admin
  ports:
```

```
- "6379:6379"
volumes:
- ./redisdata:/data
networks:
- redis
```

Конфигурируем sentinel. Вручную впишем ip redis-master, поскольку sentinel не хотел резолвить хост redis-master. Также определяем, как будем называть мастера – test_master0:

```
port 26379
sentinel monitor test_master0 172.21.0.2 6379 1
sentinel auth-pass test_master0 admin
sentinel down-after-milliseconds test_master0 5000
sentinel failover-timeout test_master0 60000
sentinel parallel-syncs test_master0 1
```

Конфигурируем компоненты в статической конфигурации userver, указывая, где искать конфигурацию для подключения к redis:

```
key-value-database:
  groups:
    - config_name: config_armai
      db: db_armai

  subscribe_groups:

  thread_pools:
    redis_thread_pool_size: 8
    sentinel_thread_pool_size: 1

  secdist:

  default-secdist-provider:
    config:
/workspaces/armai/configs/etc/redis_service/secure_data.json
    missing-ok: false
    # environment-secrets-key: SECDIST_CONFIG
```

Определяем эту конфигурацию в файле secure_data.json. Здесь мы указываем, как называется мастер:

```
{
  "redis_settings": {
    "config_armai": {
      "password": "admin",
      "sentinels": [
        {"host": "redis-sentinel", "port": 26379}
      ],
    },
  },
}
```

```

        "shards": [
            {"name": "test_master0"}
        ]
    }
}

```

Добавляем компоненты для работы с redis от userver:

```

.Append<userver::components::Secdist>()
    .Append<userver::components::DefaultSecdistProvider>(
)
    .Append<userver::components::Redis>("key-value-
database")

```

Определяем свой компонент для получения репозитория для работы с redis. Он определяет логику создания репозитория:

```

class RedisRepositoryComponent : public userver::components::ComponentBase {
public:
    static constexpr std::string_view kName = "redis-repository-component";

    RedisRepositoryComponent(
        const userver::components::ComponentConfig& config,
        const userver::components::ComponentContext& component_context
    ) : ComponentBase(config, component_context),
        redis_repository_(std::shared_ptr<repositories::redis::RedisRepository>(
            new repositories::redis::RedisRepository(
                component_context.FindComponent<userver::components::Redis>("key-
value-database").GetClient("db_armai"),
                userver::storages::redis::CommandControl(std::chrono::seconds{15}
, std::chrono::seconds{60}, 4),
                config.As<armai::infrastructure::repositories::redis::RedisConfig
>())
            )
        )) {}

    std::shared_ptr<repositories::redis::RedisRepository> GetRedisRepository()
const;

    static userver::yaml_config::Schema GetStaticConfigSchema();

private:
    std::shared_ptr<repositories::redis::RedisRepository> redis_repository_;
};

```

Также реализуем для этого компонента метод GetStaticConfigSchema, чтобы можно было задавать кастомные параметры в статической конфигурации:

```

userver::yaml_config::Schema RedisRepositoryComponent::GetStaticConfigSchema() {

```

```

        return
userver::yaml_config::MergeSchemas<userver::components::ComponentBase>(R"(
type: object
description: user component RedisRepositoryComponent
additionalProperties: false
properties:
    ttl-refresh:
        type: integer
        description: time to live refresh token in redis
)");
}

```

После этого сможем определить в конфигурации параметр, отвечающий за время жизни refresh токена в redis – задаем месяц:

```

redis-repository-component:
    load-enabled: true
    ttl-refresh: 2592000000 # 30 дней (в мс)

```

Чтобы распарсить эту конфигурацию, необходимо реализовать специальный метод Parse:

```

RedisConfig Parse(const userver::yaml_config::YamlConfig& yaml,
userver::formats::parse::To<RedisConfig>) {
    return RedisConfig{std::chrono::milliseconds(yaml["ttl-
refresh"].As<long>())};
}

```

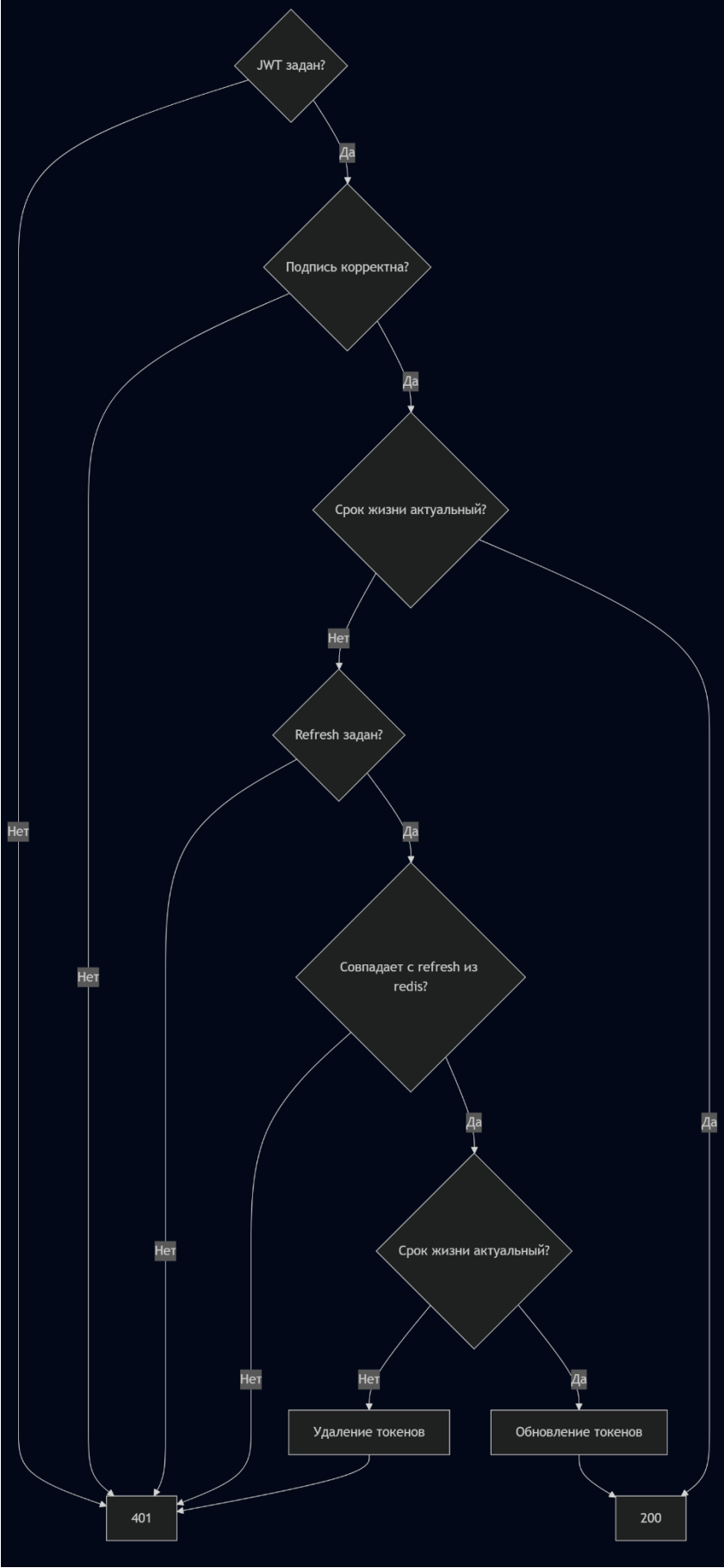
Инфраструктура полностью настроена, можно реализовывать логику. В redis репозитории реализуем следующие методы для работы с refresh токенами:

```

void SaveRefresh(const int userId, const std::string& refresh);
std::optional<std::string> GetRefresh(const int userId);
void DelRefresh(const int userId);

```

Логiku проверки авторизации пользователя вынесем в отдельный метод. Опишем ее в виде диаграммы:



Реализация в коде:

```
domain::utils::jwt::Claims checkAuth(
    const userver::server::http::HttpRequest& request,
    userver::server::http::HttpResponse& response,
    std::shared_ptr<armai::infrastructure::repositories::redis::RedisRepository>
redisRepository
) {
    const auto jwt = utils::auth::getToken(request, kJwtCookieName);
    if (!jwt.has_value()) {
        throw domain::utils::exceptions::auth::JwtNotSetException();
    }

    domain::utils::jwt::Claims claims;
    try {
        claims = domain::utils::jwt::decodeJwt(jwt.value());
    } catch (std::exception&) {
        throw domain::utils::exceptions::auth::InvalidJwtException();
    }

    try {
        domain::utils::jwt::verifyTtlJwt(claims);
    } catch (domain::utils::exceptions::auth::LifetimeExpiredException&) {
        const auto refresh = utils::auth::getToken(request, kRefreshCookieName);
        if (!refresh.has_value()) {
            throw domain::utils::exceptions::auth::RefreshNotSetException();
        }

        const auto refreshRedis = redisRepository->GetRefresh(claims.userId);
        if (!refreshRedis.has_value() || refresh.value() != refreshRedis.value())
        {
            throw domain::utils::exceptions::auth::InvalidRefreshException();
        }

        redisRepository->DelRefresh(claims.userId);
        auth(claims.userId, claims.isAdmin, response, redisRepository);
    }

    return claims;
}
```

Refresh токены в нашей системе будем считать одноразовыми. Если у пользователя есть протухший access токен и корректный refresh – генерируем новые access и refresh токены, обновляем refresh токен в redis.

Генерация токенов и установка их в cookies вынесена в отдельный метод:

```
void auth(
```



```

        const int userId, const bool isAdmin,
        userver::server::http::HttpResponse& response,
        std::shared_ptr<armai::infrastructure::repositories::redis::RedisRepository>
redisRepository
    ) {
        const auto timePoint =
std::chrono::time_point_cast<std::chrono::milliseconds>(std::chrono::system_clock
::now());
        const auto claims = domain::utils::jwt::Claims{userId, isAdmin, timePoint};
        const auto jwt = domain::utils::jwt::createJwt(claims);
        response.SetCookie(utils::auth::buildCookieToken(utils::auth::kJwtCookieName,
jwt));

        const auto refresh = domain::utils::refresh::getRefresh(userId);
        redisRepository->SaveRefresh(userId, refresh);
        response.SetCookie(utils::auth::buildCookieToken(utils::auth::kRefreshCookieN
ame, refresh));
    }

```

Таким образом, при регистрации и авторизации происходит вызов метода auth, при обращении к внутренним страницам сервиса – checkAuth.

Для выхода из системы нужно удалить токены из базы и из cookie:

```

redisRepository->DelRefresh(claims.userId);

        response.SetCookie(
            utils::auth::buildCookieToken(utils::auth::kJwtCookieName, "")
            .SetExpires(std::chrono::system_clock::time_point(std::chrono::system
_clock::now()))
        );
        response.SetCookie(
            utils::auth::buildCookieToken(utils::auth::kRefreshCookieName, "")
            .SetExpires(std::chrono::system_clock::time_point(std::chrono::system
_clock::now()))
        );

```

3 Выводы

В ходе лабораторной работы я реализовал интеграцию своего проекта на с++ с использованием асинхронного фреймворка userver с базой данных redis. С использованием redis я реализовал безопасную систему авторизации через access и refresh токены. Использование redis значительно ускорит работу моего сервиса, поскольку работа с refresh токенами – очень частая операция, (проверка авторизации происходит на каждой странице), а redis, в отличие от того же postgresSQL, хранит данные прямо в оперативной памяти.